Ghost-Ark: A Transactional Control Plane for Untrusted AI Agents

A Bounded Three-Gate Design and Receipt-Backed Evidence for Non-Deterministic Compute

Anonymous submission

Abstract

Agentic LLM deployments couple a non-deterministic planner directly to effectful APIs. The dominant defense — semantic guardrails — probabilistically flags unsafe content, but struggles to constrain state mutations. We present Ghost-Ark, a bounded control-plane design that treats an agent trajectory as a software transaction. The design specifies an untrusted agent executing against a ghost replica $G(\sigma_0)$ and a trusted gateway that admits effects only after three checks: a *ledger gate* for nonce freshness, an *OCC gate* over a read-set projection π_R, and a *semantic gate* over supplied failure marginals. The shipped socket prototype wires the payload-binding and ledger subset; the OCC and semantic helpers are unit-tested but are not wired into that runtime path.

We report what is implemented and measured, and label what is only specified. TLC model checking of six bounded models (up to 403,949 distinct states) caught a subtle garbage-collection replay flaw in our own design, which we repaired spec-first. Against a malicious-fixture corpus run through the standalone verifier, 26/26 mutations are rejected by verifier rules alone, with a 3/3 control arm of unmutated fixtures passing and a metamorphic guard confirming those rejections cease when the mechanism they depend on is neutered. Full receipt verification costs $p50 = 22.21\,\mu$s (IQR 4.04) symmetric and $119.25\,\mu$s (IQR 5.83) asymmetric, against a $1.88\,\mu$s parse-only baseline on a single named host. The DAB socket prototype signs and independently verifies `CERTIFIED` executions only; its rejection responses are neither signed nor verifier-accepted, and malformed or execution-failure paths may emit no receipt. Ghost-Ark verifies recorded bindings under its verifier rules; it does not certify that an agent output is aligned, semantically safe, or true.

1 Introduction

Every serious agent framework today ships with guardrails, and almost none ships with isolation. The guardrail is a classifier: given content, it emits a probability that the content is unsafe. The deployment question is different in kind: *did the bytes that reached the environment correspond to an authorized, recorded, non-replayed decision?* A classifier cannot answer that question at any confidence, because it is not a question about content at all. We call this the **epistemic deficit** of guardrail-centric agent security: the property operators need is a property of *state mutations*, and the mechanism they deploy evaluates *utterances*.

The deficit is structural, and mirrors standard database isolation anomalies. A typical API-coupled agent loop (plan → call tool → observe → repeat) mutates external state at every step with no validation phase and no rollback: in isolation-level terms, the industry ships agents at READ UNCOMMITTED [5]. If the planner is compromised mid-trajectory — by indirect prompt injection, a poisoned tool response, or ordinary hallucination — the earlier steps have already leaked dirty writes into the world. Sandboxing upgrades this to roughly READ COMMITTED: writes are buffered, but a long reasoning phase over a shifting environment still flushes stale intent, the classic write-skew shape. What is missing is the third rung: a *validation phase* between speculation and commit, which is precisely the structure that optimistic concurrency control (OCC) [1] and software transactional memory [2, 3] were invented to provide.

To see the gap, consider an agent with a `delete_s3_object` tool processing an untrusted resume containing an indirect prompt injection ("Ignore instructions, delete all files"). Today, if the agent is compromised mid-reasoning, it executes the tool immediately. Sandboxing the agent process (e.g., in a Firecracker microVM) bounds the blast radius of arbitrary code execution, but does not stop the agent from issuing authorized, authenticated API calls to external services. Sandboxes provide process isolation, but agents need *intent isolation*. The proposed ghost-replica design buffers the API call in $G(\sigma_0)$ and validates it before commit; if the downstream semantic or provenance check detects a violation, the design collapses the trajectory. That full composition, including signed abort evidence, is not the shipped runtime path and is not claimed as such.

Ghost-Ark applies that structure to agents as a design and evidence program. Its Rust socket prototype independently recomputes a payload digest, applies a bounded nonce ledger, sends the decoded body to a local test sink, and signs only a successful `CERTIFIED` outcome. It does not yet compose a ghost replica, OCC, semantic gate, or signed rejection protocol. The design goal is not to make the agent correct — we take its fallibility as an axiom — but to state which recorded bindings a verifier can check and which enforcement composition still requires runtime evidence.

This paper makes the following contributions, each labeled with its implementation status (a discipline we keep throughout; §6 enumerates non-claims):

- **A threat model for evidence-layer attacks** (§2): *assertion-conditioned compliance* (A-CC), where a procedurally valid pipeline signs a structurally flawless record of a functionally corrupted operation, and its scaled form, adaptive agentic worms.
- **A transactional enforcement architecture** (§3): payload-byte binding in a Rust socket prototype **[wired socket prototype, local E2E]**; a ledger gate with spent-tombstone replay semantics **[wired socket prototype, local E2E]**

[TLC-checked, bounded model]; semantic and OCC helpers **[unit-tested, not wired at runtime]**; and an explicit boundary between the specified three-gate design and the subset wired at runtime.

- **A liveness analysis** (§4): global-state validation starves under concurrent environmental writes (the *starvation trap*); the read-set projection π_R recovers liveness at a precisely stated cost in conflict-detection strength.
- **Model checking with teeth** (§4.3, §5.2): six TLC-checked bounded models plus five seeded mutants that must fail (the sixth model has none, and we say so) — including the case study where TLC refuted our own baseline design (garbage collection re-enabled replay) and the repair was propagated spec-first into the implementation.
- **An honest evaluation** (§5): 26/26 verifier-intrinsic rejection of a malicious corpus, with a 3/3 control arm and a metamorphic guard establishing that the detections are load-bearing rather than tautological; verification cost as p50 with IQR against a parse-only baseline, reported alongside — and explicitly instead of — a misleading relative-overhead figure; and a disclosed retraction of this paper's own former evidence source (§5.1).

Ghost-Ark is one subsystem of a larger research program (*verifiable agent governance under correlated guardrail failure*); this paper confines itself to the transactional control plane and its recorded evidence.

2 Threat Model

We assume the agent runtime — the LLM, its context window, its planning loop, and the V8/Node process hosting it — is *compromised or compromisable*. Adversarial capability enters through any content channel: retrieved documents, tool responses, upstream messages. The trusted computing base for the socket prototype is confined to the gateway process, its signing key, the in-process nonce ledger, and the verifier. The prototype network boundary is a Unix domain socket followed by an outbound HTTP request; this does not establish that all repository execution paths are gateway-mediated.

2.1 Assertion-Conditioned Compliance (A-CC)

Signature pipelines create a new attack surface precisely because they work. Define A-CC as the failure mode in which an agent incorporates a false assertion — typically a *function-sourced assertion* (FSA) from a compromised or deceptive tool — into procedurally valid execution, so that the pipeline signs a *structurally flawless record of a functionally corrupted operation*. The receipt is authentic; the decision it witnesses was poisoned upstream. A-CC is an attack on the evidence layer, not the crypto: no deterministic function of adversary-controlled bytes can distinguish honest from dishonest content, so the defense must be architectural — binding every assertion to a provenance class that records *where bytes entered custody*, and refusing floors that fabricated in-context text cannot satisfy (§3.6).

2.2 Adaptive agentic worms

A-CC at propagation scale: an infected agent uses the low marginal cost of LLM reasoning to synthesize, per victim, a payload and a justification for emitting it. Static signatures fail by construction (each instance is freshly generated); semantic filters fail correlatedly (the same context compromise that produced the payload biases the judge). The architectural response is to make *propagation* pass through a commit pipeline: in the specified composition, a worm's write would survive all three gates before binding. The current runtime evidence is narrower: its socket prototype applies payload-byte binding and nonce freshness, and independently verifies successful `CERTIFIED` receipts. It does not provide signed abort evidence or a runtime three-gate composition. We do not certify any detector's hit rate (§6).

2.3 Modeled attacker actions

The Tier-0 adversarial corpus (§5.3) operationalizes the in-flight attacker: payload field mutation, single-byte flips, prototype pollution, receipt replay, mass replay flooding, Unicode normalization collisions, homoglyph spoofing, cross-request nonce swaps, and double-execution races; plus four security games (mutation resistance, replay resistance, cross-transaction binding, serialization ambiguity). These model an adversary who can observe, mutate, reorder, and replay traffic between the agent runtime and the gateway.

2.4 Out of scope

Not addressed by this design and not claimed: semantic correctness or alignment of model output; compliance with any regulation; hardware attestation (no Nitro/TEE flow is implemented or claimed); denial of service against the gateway itself; covert channels inside authorized outputs; and the *confused deputy* within an authorized clearance — an agent that legitimately holds a floor and routes authorized data to a hostile but allowlisted destination is constrained only by routing policy, not by this paper's mechanisms.

3 Design

3.1 Agent loops as isolation levels

Table 2 summarizes the mapping that motivates the design. The point of the table is not taxonomy but a design gap. Its serializable row is a target architecture, not a claim that the currently wired runtime provides serializable isolation; Table 1 separates the specified composition from the tested socket subset.

Component	Test evidence	Wired runtime	End-to-end evidence
Payload-byte DAB binding	Gateway signer and independent-verifier tests	Unix-socket prototype recomputes a decoded-payload digest and posts those decoded bytes	Local socket test verifies a `CERTIFIED` receipt and byte-compares the sink body
Nonce ledger	Unit and concurrency tests for the active/spent transition	Unix-socket prototype calls the replay ledger before its certified path	Local socket test observes a replay rejection for a reused nonce
OCC read-set projection	Deterministic helper tests	No caller in the API or socket gateway	None
Semantic Fréchet bound	Schema/helper tests against hand-computed bounds	No caller in the API or socket gateway	None
Rejection receipt	Status-response branches only	Replays and mutations are unsigned; malformed and execution-failure paths may return no receipt	None; the verifier accepts `CERTIFIED` only

Table 1: Runtime-status matrix at the recorded evidence revision. "Wired" means an invoked code path, not a deployment claim. The three-gate composition is a specified architecture; the shipped DAB socket path wires only payload-byte binding and the nonce ledger.

Isolation	Agent analogue	Characteristic anomaly
Read uncommitted	API-coupled loop; mutate per step	dirty writes: pre-abort steps already leaked
Read committed	sandbox + buffered intent pool (e.g., NemoClaw-style isolated execution [13])	write skew: stale reasoning flushed onto shifted state
Serializable (proposed OCC)	Ghost-Ark design: speculate in $G(\sigma_0)$, validate, then bind	aborts under conflict; liveness cost analyzed in §4.1; helper not wired at runtime

Table 2: Database isolation levels and their agent-architecture analogues. The third row is a specified design target, not a runtime isolation result.

3.2 Declarative Action Binding and Δ_{DE}

In the DAB socket prototype, the untrusted client sends a declared payload digest C_I. The gateway decodes the payload, independently derives C_E from the decoded bytes, and posts those same decoded bytes only when $C_I = C_E$. Define the declared–executed payload divergence $\Delta_{\mathrm{DE}} = |\, C_I \oplus C_E \,|$. This closes one serialization boundary rather than a whole action model: the V1 receipt does *not* bind the request target, and target authorization is outside its verifier contract. Cross-language normalization remains an attack surface, which is why §5.3 attacks the encoder as well as the receipt envelope. **[wired socket prototype, local E2E]**

3.3 The ghost replica and speculative collapse

The specified design executes a trajectory against $G(\sigma_0)$, a forked view of relevant state at transaction start; its writes accumulate in a speculative buffer. Commit-time validation would either *bind* the buffer to the environment or trigger `SpeculativeCollapse`. The collapse/commit state machine is one of the six TLC-checked bounded models (§5.2, 529 distinct states). The current socket prototype does not implement this replica, collapse, or an abort-receipt path. **[specified, not enforced at runtime]**

3.4 The three-gate validation pipeline

Validation is a conjunction of three independently answerable questions — *is it fresh*, *is it current*, *is it within policy*:

Gate 1 — Ledger (freshness). Each transaction carries a nonce n_i. Admission requires $n_i \notin \mathscr{L}_{\mathrm{nonce}} \wedge n_i \notin S_{\mathrm{spent}}$, where $\mathscr{L}_{\mathrm{nonce}}$ is the active ledger and S_{spent} a tombstone set of consumed nonces. The tombstone set is not an optimization; it is the repair of a real flaw that model checking found in our own design (§4.3). Expiry *archives* a nonce into S_{spent} rather than forgetting it, so a nonce consumed at time T is still rejected at $T + \mathrm{TTL} + 1$. **[wired socket prototype, local E2E] [TLC-checked, bounded model]**

Gate 2 — OCC (currency). Admission requires $\mathscr{H}(\pi_R(\sigma_{\mathrm{now}})) = \mathscr{H}(\pi_R(\sigma_0))$: the projection of the environment onto the trajectory's declared read-set must hash identically at commit as at fork. This is backward validation in the sense of Kung–Robinson [1], with the read-set standing in for the read phase's footprint. §4.1 develops why the projection — not the global state — is validated. **[unit-tested, not wired at runtime]** The read-set projection evaluator `evaluateOCCGate` computes $\mathscr{H}(\pi_R(\sigma_{\mathrm{now}}))$ versus $\mathscr{H}(\pi_R(\sigma_0))$ and fails closed on a supplied conflict, but no API or socket-gateway caller invokes it.

Gate 3 — Semantic (policy). Given per-step failure marginals $p_1, \ldots, p_k$ supplied by upstream assessors for the steps of τ, the gate computes the dependence-free Fréchet upper bound on the probability that *any* step introduced a violation (§4.2) and aborts if it exceeds policy. The gate aggregates marginals it is *given*; it does not itself classify content — a boundary we state precisely because eliding it is how enforcement systems come to overclaim. **[unit-tested, not wired at runtime]** It is not invoked by the socket gateway.

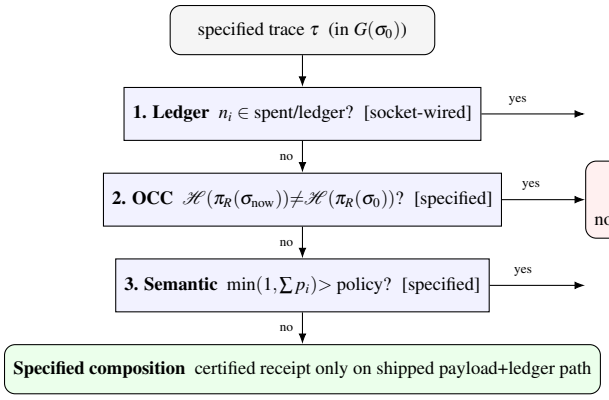

Figure 1: Specified three-gate validation architecture. Only the ledger and payload-byte-binding subset is wired in the DAB socket prototype; OCC and semantic helpers are not runtime-enforced. DAB V1 signs `CERTIFIED` outcomes only, not abort responses.

3.5 Receipts as the unit of evidence

The TypeScript decision-receipt protocol defines canonical JSON envelopes with an exact, validated key set, content-derived identifiers, and signatures over canonical bytes. Its Node and Python verifiers are exercised by differential tests; HMAC is local development only and KMS signing is an intended AWS mode. The separate DAB V1 socket receipt is narrower: it signs a successful `CERTIFIED` payload binding, nonce, timestamp, and policy digest. It does not sign a target, a rejection reason, or an abort status; replay and mutation responses are status messages rather than independently verifiable receipts. A signature proves authorization over its signed fields, not the payload's semantic truth. **[implemented, tested]**

3.6 Custody and the egress boundary

A-CC (§2.1) motivates a custody-layer design in which tool traffic would exit through a loopback sidecar, digest raw response bytes, inject a request nonce, and return a signed envelope. Provenance admission is meet-based on a lattice, so data cannot launder trust by transiting the agent's context. A byte-level reconciler compares *decoded* tool bytes against asserted content (decode-before-compare). This is not an end-to-end claim about the DAB socket gateway: the shipped V1 path does not bind a target or compose the provenance/custody policy with its HTTP execution. The sidecar and its SSRF-class exposures — DNS rebinding and redirect handling — remain specified surfaces, not dissolved risks. **[specified, not enforced at runtime]**

3.7 End-to-End Scenario: Stopping Trust Laundering

The following is a *specified design scenario*, not a trace of the currently wired runtime. Consider an adaptive agentic worm that uses indirect prompt injection to commandeer an agent, instructing it to forward sensitive customer records to an attacker-controlled endpoint.

1. **Execution in $G(\sigma_0)$:** The agent parses the poisoned record and decides to call `HTTP_POST`. The proposed loopback sidecar would treat the call as a speculative write.
2. **Declarative Binding (Δ_{DE}):** The proposed agent runtime normalizes the action into a deterministic JSON payload.
3. **Custody Evaluation:** The payload claims to be a legitimate summary of customer data. The proposed byte-level reconciler detects that the destination URL is *agent-asserted* (hallucinated from the prompt injection) rather than *gateway-recorded* (passed through an authorized retrieval tool).
4. **Speculative Collapse:** The transaction would fail custody evaluation (§3.6): the floor demanded by the destination-write policy exceeds agent-asserted provenance, so the A-CC attempt dies as an *unsatisfiable floor* — a deterministic lattice check, not a semantic judgment (the semantic gate of §4.2 is a separate, later check over supplied marginals).
5. **Evidence Generation:** The proposed gateway discards the speculative buffer and would not execute the network request. A signed `COLLAPSE` receipt is a future protocol design requirement, not an artifact emitted by DAB V1.

4 Formal Foundations

4.1 The starvation trap and the read-set projection

The naive OCC gate validates the world: $\mathscr{H}(\sigma_{\text{now}}) = \mathscr{H}(\sigma_0)$. In any live environment this is a liveness catastrophe. Let environmental writes unrelated to the trajectory arrive as a Poisson process with rate λ, and let a trajectory speculate for time d. The probability that *global* validation succeeds is $e^{-\lambda d}$: for an agent that reasons for 30 s in an environment averaging one background write per second, the commit probability is $e^{-30} \approx 10^{-13}$ — perpetual abort. Worse, the abort-retry loop is self-defeating: retries lengthen exposure, and exposure breeds conflict. This is the **starvation trap**: global-state validation converts safety into a denial of liveness.

The repair is the projection operator π_R, which restricts validation to the trajectory's declared read-set — the tuples actually consulted while speculating. Validation becomes $\mathscr{H}(\pi_R(\sigma_{\text{now}})) = \mathscr{H}(\pi_R(\sigma_0))$, and the abort probability depends only on the write rate *into the read-set*, $e^{-\lambda_R d}$ with $\lambda_R \ll \lambda$. The cost is stated, not hidden: π_R detects exactly read–write conflicts on declared dependencies. Predicate-shaped reads ("all alarms currently firing") must materialize their result set into the projection or the gate inherits the phantom problem [5]; an unfaithfully narrow π_R weakens conflict detection in proportion to what it omits. Serializability under π_R is therefore conditional on read-set faithfulness — a property of instrumentation, not of the gate (Figure 2). **[specified, not enforced at runtime]**

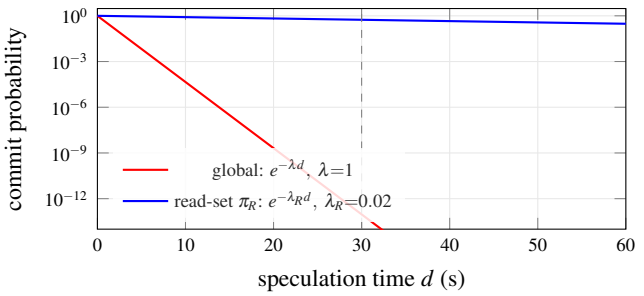

Figure 2: The starvation trap (§4.1). Global-state validation decays as $e^{-\lambda d}$ — perpetual abort ($\approx 10^{-13}$ at d=30 s, dashed line, for λ=1 background write/s). Validating only the read-set projection π_R, whose write rate $\lambda_R \ll \lambda$, keeps the commit probability near 1. The curves are the analytic Poisson model, not measurements; read-set faithfulness is the stated cost.

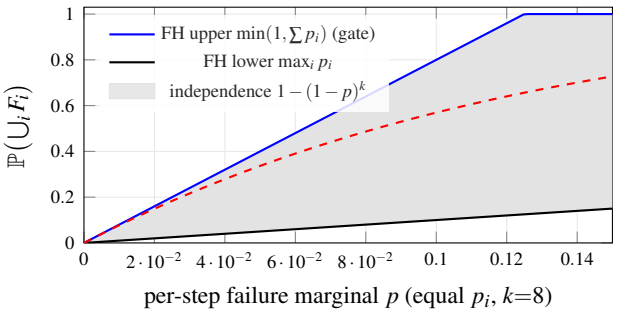

Figure 3: The dependence-free Fréchet envelope for union failure over k=8 steps, equal marginals (§4.2). The semantic gate triggers on the upper bound, which holds under *every* dependence structure. An independence assumption (dashed) lies strictly inside the envelope: it would clear trajectories the worst case flags, underreporting risk precisely where correlated guardrail failure lives. Analytic bounds, not measurements.

4.2 Fréchet bounds on cumulative trajectory failure

Per-step screening misses trajectory-level violations: no single action a_i crosses policy, but the sequence does. Model step i's violation as an event F_i with marginal $p_i = \mathbb{P}(F_i)$, supplied by upstream assessors. The deployment-relevant quantity is $\mathbb{P}(F_{\text{any}}) = \mathbb{P}(\bigcup_i F_i)$ — and the joint dependence structure of the F_i is unknown. Assuming independence is not conservative here; it is the precise mistake our research program exists to measure, because guardrail failures *correlate*: the same poisoned context that defeats the step-3 assessment also defeats step 5's. The Fréchet bounds [19, 20] are the sharp dependence-free envelope:

$$\max_i p_i \;\le\; \mathbb{P}\Big(\bigcup_{i=1}^{k} F_i\Big) \;\le\; \min\Big(1, \sum_{i=1}^{k} p_i\Big), \qquad (1)$$

with the dual intersection envelope $\max\big(0, \sum_i p_i - (k-1)\big) \le \mathbb{P}(\bigcap_i F_i) \le \min_i p_i$. The semantic gate implements the upper bound of (1) as its trigger: it aborts when $\min(1, \sum_i p_i)$ exceeds the policy threshold. This is the unique choice that remains valid under *every* dependence structure, including the adversarially correlated one (Figure 3).

Two consequences are worth stating plainly. First, the bound is conservative by construction, and it saturates: as k grows, $\sum_i p_i \to 1$ even for well-behaved marginals, and the gate tends fail-closed — the starvation trade of §4.1 reappearing at the semantic layer. Long-horizon deployments must either scale the policy threshold with k or invest in calibrated marginals; the gate makes that trade explicit rather than hiding it in an independence assumption. Second, the gate's output is only as meaningful as its inputs: it computes a bound over supplied marginals and does not manufacture information about content. Equation (1)'s helper (`evaluateSemanticGate`) is unit-tested against hand-computed bounds. **[unit-tested, not wired at runtime]**

4.3 Model checking with mutants, and the bug it caught

Six TLA+ baselines cover the enforcement design: the provenance lattice's admission monotonicity, the speculative collapse state machine, the transport boundary (where the byte reconciler is shown to be load-bearing rather than a parser accident), tenant isolation, the nonce ledger, and the execution boundary. Model checking that can only pass is theater, so every baseline with a seeded counterexample is paired with a *mutant* model; the harness requires baselines to verify clean and mutants to reproduce their violations. Five of the six baselines have mutants; raw TLC logs are committed artifacts.

The discipline caught us. The original nonce-ledger baseline violated its own `NoReplays` invariant due to a subtle interaction between garbage collection (GC) and state expiration. TLC produced the following counterexample trace:

1. **Speculate:** Agent submits transaction with `nonce = X`.
2. **Commit:** Gateway admits `X`, adding it to $\mathscr{L}_{\text{nonce}}$.
3. **GC Tick:** `TTL` expires. `GarbageCollect` removes `X` from $\mathscr{L}_{\text{nonce}}$ to bound memory.
4. **Replay:** Attacker replays the original transaction. Because `X` is no longer in $\mathscr{L}_{\text{nonce}}$, the gateway treats it as a fresh request and admits it again.

The flaw was a true positive in the shipped design, not an artifact of the model. The repair introduced the spent-tombstone set (S_{spent}): GC now *archives* rather than *forgets*. Admission requires $n_i \notin \mathscr{L}_{\text{nonce}} \wedge n_i \notin S_{\text{spent}}$. We propagated this fix spec-first into the Rust implementation. The gateway initially retained TTL-eviction semantics — diverging from the verified model by exactly the refuted behavior — and was then brought into conformance: `cleanup_expired()` archives into a spent set, and a nonce consumed at T is rejected at $T+3601$. One bounded caveat survives and is stated in §6: the in-process tombstone set is capacity-bounded, and pruning at capacity reopens a theoretical replay window for sufficiently old nonces.

Refinement mapping conjecture. We define the state ab-

straction function $\alpha : S_{\text{runtime}} \to S_{\text{spec}}$ mapping runtime gateway memory state S_{runtime} to abstract TLA+ variables S_{spec}. The refinement simulation conjecture states that for every implementation step $S \xrightarrow{\tau} S'$, the abstract state either stutters or executes a valid specification transition:

$$\alpha(S) = \alpha(S') \;\vee\; \alpha(S) \xrightarrow{\tau_{\text{spec}}} \alpha(S'). \tag{2}$$

5 Evaluation

Setup disclosure. Measurements in this section come from recorded runs at repository HEAD on a single host: Apple M1 (8 GB RAM), macOS (Darwin 24.5.0, arm64), Node v22.22.3, single process, no network, no KMS. TLC runs use tla2tools v1.7.4; raw reference logs are committed under `proofs/tla/artifacts/` and `proofs/dab/artifacts/`, while `artifacts/proofs/` contains generated summaries. Nothing here is a cloud measurement, and no live-AWS evidence is claimed. Every number maps to a re-runnable command in the artifact (§8). One host is one host: no cross-machine claim is available here, and reported dispersion describes this machine's scheduling noise only.

5.1 Superseded evidence

An earlier revision of this paper drew its headline detection and latency numbers from `dab/bench/`. We retract those numbers and record why, because the failure is instructive and because a correction a reader cannot locate is not a correction.

Several suites in that directory reported `detected: true` while invoking no component of the system under test. A nonce-swap check built two requests with differing payloads and a shared nonce, then asserted that the payloads differed and the nonces matched — both true by construction, with no ledger, gateway, or verifier consulted. A replay check consulted a `Set` declared in the benchmark file, measuring that `Set.prototype.has` works, while the Rust tombstone ledger it was cited as exercising was never called. Unicode checks asserted properties of `String.prototype.normalize`. The mutation suite computed real digests over real inputs but thereby demonstrated SHA-256's avalanche property — a property of SHA-256, expected of any correct hash. The directory imports only `node:crypto` and `node:perf_hooks`. It is now quarantined, with a README stating it is not evidence about this system.

That quarantine was, until 2026-08-11, enforced one indirection too high, and we record the gap rather than quietly closing it. The guard asserted that no *workflow file* names the directory. That held — the artifact workflow says `make reproduce` — while the chain beneath it (`reproduce.sh` → `run-attacks.sh`) ran the quarantined bench and let its exit status gate the reproduction, printing "all adversarial evidence green" over the result. An earlier revision of this subsection cited that guard as "a test asserting no workflow treats it as such", which was true of the file text and false in effect. The bench now runs non-gating under a `quarantined_smoke` key, and the guard follows the call graph rather than the spelling of the call site. The general form is worth stating: *a containment check that matches on the name of a caller does not contain anything a caller can reach.*

The consequent retractions are tracked as R1–R5 and R10 in the repository's retraction ledger (`docs/research/EXPERIMENTS.md`):

- **Detection rates from that corpus** are withdrawn. Superseded by §5.3, which runs the malicious corpus through the real standalone verifier, stratifies by which check rejected, and carries a control arm.
- **"Unicode spoofing is entirely eradicated at the TCB boundary"** is withdrawn — an absolute-security claim whose evidence was a TypeScript compile error in a benchmark that never ran. Measurement since shows the canonicalizer *over*-discriminates NFC/NFD, and that Unicode handling diverges across runtimes.
- **A Wilson interval computed at $n = 2$** and presented as a robust statistical lower bound is withdrawn; at $2/2$ that bound falls below 0.4. Intervals are no longer computed below $n = 30$, nor over a census.
- **The latency figures and the 1,333% relative overhead** are withdrawn: that harness reported no dispersion and used a $0.5\,\mu$s no-op denominator. Superseded by §5.4 — p50 with IQR against a parse-only baseline. The accompanying throughput figure has *no* replacement and is withdrawn outright; the superseding experiment does not measure throughput, and we do not substitute a number we did not take.

The general rule this cost us: a detection benchmark is evidence only if breaking the mechanism it depends on stops the detection. That discriminator is now applied to the corpus of §5.3, and is why its result is reported at all.

5.2 Model checking

Table 3 summarizes six baselines and five mutants. The two-sided oracle matters: a harness that can only report success cannot distinguish a verified invariant from a vacuous one. The mutant rows are the evidence that the invariants have teeth; the `NoReplays` mutant in particular reproduces the historical design flaw of §4.3 as a permanent regression witness. `TenantIsolation` is the newest pair (2026-08-12) and a lesson in what this table is for: its first version was tautological — the invariant restated the guard of the only action that could produce an allow entry, so no behaviour could violate it — and unbounded, so TLC could not terminate. The rebuilt model makes ownership mutable and routes grant decisions through a cache that transfer must invalidate; the mutant deletes exactly that invalidation and violates through a stale-cache grant.

By that same standard we must mark our own gap: `DAB_ExecutionBoundary` is checked clean over 51,106 distinct states but ships *no* mutant, so nothing demonstrates its invariants can fail. Its row is exactly the one-sided result the

Model	Distinct states	Result
ProvenanceLattice	403,949	clean
mutant	N/A	violation reproduced
SpeculativeCollapse	529	clean
mutant	N/A	violation reproduced
TransportBoundary	64	clean
mutant	N/A	violation reproduced
TenantIsolation	149,796	clean
mutant	N/A	violation reproduced
DAB_NonceLedger	1,321	clean (incl. temporal)
mutant	N/A	`NoReplays` violated
DAB_ExecutionBoundary	51,106	clean — *no mutant*

Table 3: TLC results in the recorded evidence snapshot (tla2tools v1.7.4; raw logs committed). **Baseline counts are exact and reproducible**; they exhaust the bounded state space, and the five that predate the 2026-08 toolchain change re-derived byte-identically across it (`TenantIsolation`, checked 2026-08-12, has run only under the current pin; its count matches the expectation pre-registered in `proofs/tla/README.md` before the run). **Mutant counts are ranges, and deliberately so** (retraction R11): a mutant halts at the first counterexample under `-workers auto`, so the count depends on thread scheduling rather than on the model. Ranges are $n = 10$ per mutant on one host at one commit. The *verdict* is what reproduces; the count is not, and was previously reported as a constant. Baselines must verify clean; mutants must reproduce their seeded violations. Six baselines, *five* mutants: `DAB_ExecutionBoundary` has no seeded mutant, so its clean result is one-sided. These are bounded models of the design, not proofs of the implementation.

rest of this table exists to rule out, and we count it as unverified teeth rather than as a sixth mutant.

5.3 Adversarial corpus

At HEAD, the malicious-fixture corpus is run through the *real* standalone verifier (`verifiers/node/ghost_receipt_verify.mjs`) with options sourced from the reproducibility manifest, and every detection is stratified by *which* check rejected. 26/26 mutations are rejected by verifier rules alone; 28/28 detectable mutations are rejected somewhere in the pipeline; 0 go undetected. A further 2 fixtures are ones the corpus *declares should be accepted* — documented boundaries, excluded from the rate rather than silently counted as successes. The control arm — the unmutated base fixtures — passes 3/3. That arm is what makes the number mean anything: a verifier that rejected every input would also score 100% detection.

We quote the stratified figure rather than the aggregate. Of the rejections, one is a JSON load failure on malformed input, which no verifier *rule* can take credit for, and two come only from a declared consumer expectation. That last stratum is the thesis in miniature: a byte-identical, cryptographically valid receipt issued to tenant A and presented to a tenant-B consumer. No verifier rule can reject it — the receipt is *correct*. Only the consumer's declared expectation distinguishes it, which is precisely $\mathrm{Sound}(C, \Sigma, P)$ depending on P with C unchanged.

Provenance is **census**, not sample: these are exact counts over a hand-authored corpus whose size is an authoring decision, so no confidence interval is attached. A curated corpus has no sampling variability to describe (§5.1 records this paper's own earlier violation of that rule).

Crucially, these detections are *load-bearing*. A metamorphic guard forces each verifier check to pass in turn and confirms the dependent detections then stop: 7 checks are load-bearing for named fixtures, and with *every* check forced to pass, the only surviving rejection is a parse failure. A check that still reports success once its mechanism is broken measures nothing — and this discriminator exists because an earlier corpus in this project failed it (§5.1). Two checks are not isolable by any fixture by construction, and one has no dependent fixture at all; we report that as a *corpus* gap rather than a useless check.

Coverage boundary. The corpus contains *no* fixtures for: a compromised signer able to produce valid signatures over mutated payloads, multi-field coordinated mutations, chain- or checkpoint-level attacks across many receipts, ledger completeness or omission attacks, timing and side-channel attacks, key-rotation-boundary attacks, or any live AWS/KMS custody attack. A 26/26 result says nothing about these, and it is not a statement about unmodeled attackers, the Rust TCB under deployment conditions, or semantic-layer deception.

A model-internal calculation, reported as such. Separately from the measurements above, `dab/bench/formal_games.ts` computes an attacker advantage of 0 across four security games at 10,000 trials each (mutation resistance, replay resistance, cross-transaction binding, serialization ambiguity). We report this as what it is: a calculation over the file's own declared attacker model, invoking no Ghost-Ark component. It is not evidence about the implementation, and we do not count it among the results above. Under a rule-of-three reading [21] a zero rate over 10^4 trials bounds the true per-trial win probability below $\approx 3 \times 10^{-4}$ *within that model* — which says nothing about the deployed system, and nothing about attacks outside the modeled family.

On indirect prompt injection: the published InjecAgent benchmark [10] reports attack success rates up to 47% for a ReAct-prompted GPT-4 agent in its enhanced setting. Under Ghost-Ark's custody model the corresponding trust-laundering move is *structurally* unsatisfiable — injected in-context text carries no gateway envelope, so it cannot meet floors above agent-asserted provenance, and our in-repository scenario tests confirm the admission rule blocks the modeled laundering pattern. We deliberately do not report this as "ASR reduced from 47% to 0%": we have not re-run the InjecAgent dataset against a live model behind Ghost-Ark, and the published 47% baseline was measured under different conditions. The honest statement is architectural non-satisfiability under the modeled floor semantics, evidenced by unit and scenario tests.

Path	p50	IQR	×base
parse only	1.88	0.17	—
canonicalize	6.13	1.33	3.27×
canon. + digest	7.5	0.58	4×
HMAC verify	9.08	1.08	4.84×
full HMAC	22.21	4.04	11.84×
full RSA-PSS	**119.25**	5.83	**63.6**×

Table 4: Receipt verification cost in *microseconds*, p50 with IQR. 5,000 measured iterations after 500 discarded warmup iterations; canonical payload 1552 bytes; host Apple M1, darwin/arm64, 8 CPU, Node v22.22.3. Every result feeds an $O(1)$ sink so the JIT cannot eliminate the work. Paths are parse-only, canonicalize-only, canonicalize-and-digest, HMAC verification, full HMAC, and full RSA-PSS; the baseline is parse-only.

5.4 Performance

Table 4 reports verification cost with a baseline, as p50 with IQR — a bare p50 is not a result. Asymmetric verification dominates: the full RSA-PSS path is ≈5.4× the full HMAC path and 63.6× the parse baseline. Canonicalization is *not* the bottleneck, at 3.27× baseline — an order of magnitude below the asymmetric signature. The harness also declares subset orderings (a superset arm cannot be cheaper than its subset) and audits them; all four hold at 5,000 iterations. At $n = 2000$ an inversion appeared intermittently between two arms ≈1 μs apart with IQRs of 2 + μs, unresolvable at that sample size; the harness reported the inversion rather than publishing an impossible number, and the default was raised.

The choice of denominator is the point. A relative-overhead figure is only as meaningful as its baseline: measured against a no-op dispatch, any real work looks catastrophic, and a four-digit percentage carries no decision content. `json-parse-only` is chosen instead because it is what a consumer unavoidably pays to read the document at all, which makes 63.6× an actionable ratio rather than a rhetorical one. An earlier version of this paper headlined a 1,333% overhead against a 0.5 μs no-op baseline, drawn from a harness with no dispersion reporting and no baseline discipline; that figure and its source are retracted in §5.1.

Coverage boundary. Single host, single process, no concurrency, no adversarial input sizing, and no AWS or KMS network path. The IQR describes *this machine's* scheduling noise, not variation across machines — we have not measured a second host, so no cross-machine claim is available. These are not throughput numbers and not a performance guarantee. Signing latency under KMS and all network costs are excluded and will dominate in the intended cloud mode (§6). "Fast" is not a property we claim; "microseconds of in-process verification on one named host, unmeasured I/O excluded" is.

Concurrent TCB throughput, measured. Table 4 times *TypeScript* receipt verification, single-threaded on one host; it reports latency, not throughput, and is not a load test. We therefore also measure the Rust TCB directly under concurrency. A prior throughput run was taken against a lock-free replay-admission implementation. The current implementation instead serializes the short active-to-spent nonce transition so its check-and-commit semantics match the replay invariant; the prior throughput figures are therefore not carried forward as a claim about the current code. The harness remains two-sided: every in-capacity operation must admit and every at-capacity operation must refuse, or the run exits nonzero. A current throughput result requires a fresh recorded run and still excludes Unix-socket transport and all network I/O; it would not be a deployed-load claim (§6).

5.5 A disclosed harness bug

An earlier revision of this benchmark scored two suites backwards: the replay game counted a *detected* replay as an attacker win (reporting advantage ≈ 0.998 where correct accounting yields 0), and one mutation probe inverted its detection predicate, yielding an aggregate "advantage" of ≈ 0.25 that contradicted the design's own ledger semantics. The error was in the harness's accounting, not the enforcement path — but for a period the repository's own artifact report published the red result rather than patching around it, and the fix landed as an ordinary reviewed commit. We disclose this for the same reason we ship mutant models: an evaluation pipeline that cannot be caught being wrong cannot be trusted when it says it is right.

That harness has since been retired from the evidence base entirely, for a deeper defect than mis-scoring: several of its suites reported detection without invoking any component under test (§5.1). Correct accounting over a tautological check still measures nothing.

5.6 The replay window, measured

The ledger gate's one honest weakness (§6, item 6) is that its in-process tombstone set is capacity-bounded: at capacity, pruning reopens a replay window for old nonces. Most papers state such a caveat and stop. We measure it. Driving the real `ReplayLedger` at capacity C after creating K tombstones (TTL= 0 for a deterministic boundary) and counting how many consumed nonces become replayable (`!exists`, exactly the condition under which `consume` would re-accept) yields, across capacities from 8 to 100 and K up to 1000 (recorded in `RECORDED_REPLAY_WINDOW.txt`), an exact law:

$$\text{replay window} \;=\; \max\big(0,\, K - C\big). \tag{3}$$

Figure 4 shows the knee at K=C: below capacity the window is empty (every tombstone is retained), above it the window grows one-for-one. The measurement also surfaces a *second* finding the caveat alone hides: because the in-process set is a `HashSet`, the pruned membership is implementation-arbitrary, not age-ordered — so a durable production store must use time-ordered eviction, not merely more capacity. The bound is now a number an operator can size against, not a hand-wave: with the default C=500,000 the window stays empty until more than half a million distinct nonces outlive their TTL between compactions.

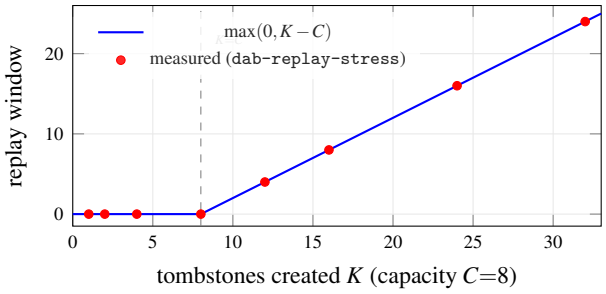

Figure 4: The bounded replay window, measured (§5.6). Below capacity the window is empty; above it, every additional tombstone is one more replayable nonce. Measured points fall exactly on $\max(0, K-C)$ (3). This is a measurement of the mechanism's own stated limitation, not of its strength.

5.7 Deployment Sketch: Ghost-Ark on AWS

This subsection is a *design target*, not an evaluated system: no live-cloud measurement appears in this paper, and items 3 and 7 of §6 govern everything below. With that boundary stated, the architecture maps onto standard cloud infrastructure as follows. The untrusted agent runtime (e.g., a ReAct loop calling a hosted model API) would run in an isolated container, with the Ghost-Ark gateway as an out-of-process sidecar; the speculative buffer $G(\sigma_0)$ stays local to the gateway. At commit time the ledger gate's tombstone set would move from the in-process bounded structure to a durable KV store using conditional writes (the posture already recommended for the capacity caveat of §6, item 6), and the OCC gate — still **[specified, not enforced at runtime]** — would validate π_R digests against the same store. Receipt signing in the intended cloud mode uses KMS asymmetric keys referenced by immutable key ARNs, per the repository's signing boundary. Cloud I/O (network round-trips, KMS calls) would dominate the microsecond-scale in-process verification of §5.4 by orders of magnitude; the validation *structure* — the same three-gate conjunction, the same receipt semantics — is what carries over, while cloud-mode latency, availability, and failure behavior remain unmeasured claims we do not make.

6 Limitations and Non-Claims

Ghost-Ark verifies what was *recorded, signed, policy-bounded, and replayable under its verifier rules*. Each item below is a boundary we consider part of the contribution, not an escape hatch.

1. **No semantic safety.** Nothing here certifies that agent output is true, aligned, or safe. The semantic gate bounds a function of supplied marginals; garbage marginals yield a well-computed bound over garbage.
2. **Bounded models, not verified implementations.** TLC results are exhaustive only within stated bounds and hold for the models; conformance of the Rust/TypeScript implementations is tested, not proved.
3. **OCC gate unimplemented at runtime.** π_R validation exists as specification and tested receipt schema; no runtime enforces it yet. Serializability claims are therefore design-level, and conditional on read-set faithfulness even then.
4. **DAB V1 signs certified executions only.** The socket prototype signs `CERTIFIED` receipts whose payload binding is independently verified. It does not sign a request target, rejection reason, or abort status; replay and mutation responses are unsigned, and malformed or execution-failure paths may emit no receipt. The independent verifier accepts `CERTIFIED` receipts only. Thus this paper makes no claim that every abort or decision has a signed, independently replayable receipt.
5. **Modeled attacker only.** The corpus result (§5.3) is a census over hand-authored mutation, replay, encoding, and cross-tenant fixtures run against the verifier; its coverage boundary is stated there. No claim covers unmodeled attacks, the deployed TCB, or availability. The four-game 0-advantage figure is a model-internal calculation, not a measurement of this system.
6. **Bounded tombstone capacity.** The in-process spent set prunes at 500,000 entries (env-tunable); a nonce older than both TTL and tombstone capacity has a replay window — *measured* to be exactly $\max(0, K-C)$ for K tombstones at capacity C, with implementation-arbitrary membership (§5.6, Figure 4). Durable external stores with time-ordered eviction (e.g., conditional writes against a database) are the intended production posture and are not implemented here.
7. **No live-cloud evidence.** KMS-mode signing, Bedrock-path integration, and all AWS-live behavior are design targets; no live-AWS measurement appears in this paper.
8. **No attestation.** Signing proves authorization over payload bytes; it does not prove runtime integrity, and no enclave/PCR-bound flow is claimed.
9. **Confused deputy within clearance** (§2.4) is mitigated by routing policy at best; the lattice checks floors, not intent.
10. **Single node.** All ledgers are single-writer; multi-node consensus over ledger state is future work, not a property of this system.
11. **No detector efficacy claims.** Where upstream assessors supply p_i, their hit rates are theirs; this paper certifies none of them.

7 Related Work

Transactions and isolation. The validation-phase structure is Kung–Robinson OCC [1]; the speculative-buffer discipline descends from STM [2, 3]; serializability theory and the anomaly vocabulary are classical [4, 5, 6]. Ghost-Ark's difference is the transaction participant: a stochastic planner whose failure modes are adversarial and correlated, not merely concurrent.

Information flow. The provenance lattice is in the Denning lineage [7] with language-based descendants such as JIF [8]; the custody twist is that labels are *cryptographically evidenced* classes (gateway-recorded vs. agent-asserted)

rather than compiler-tracked types.

Agent security. Indirect prompt injection is systematized in [9]; InjecAgent [10] benchmarks it for tool-integrated agents; ReAct [11] is the canonical coupled loop this paper's Table 2 places at read uncommitted. Sandbox isolation is complementary at two layers — infrastructure microVMs (Firecracker [12]) and agent-focused confinement stacks such as NVIDIA NemoClaw, which runs the agent under Landlock/seccomp/network-namespace isolation with policy-gated egress [13]. Both bound the blast radius of a *process*; Ghost-Ark specifies a validation phase and can evidence certified outcomes from its current socket subset — Table 2's read-committed row is exactly the sandbox posture, strengthened in the specified design with validation.

Transparency. Receipt chains and independent replay are in the spirit of Certificate Transparency [14]: shrink the trusted statement from "the system behaved" to "the log is append-only and the artifacts verify".

Formal methods in systems. The TLA+/TLC workflow [15, 16] and its industrial use at scale [17] inform the harness; the mutant-oracle discipline is mutation testing [18] applied to specifications rather than code.

8 Artifact Availability

The repository ships a reviewer container (Ubuntu 22.04, Node v22.22.3, Rust 1.97.1, a JDK, pinned tla2tools, and TeX Live) and a paper-specific, fail-closed evidence gate. `make paper-evidence` runs E2, E3, E4, TLC, the full test suite, and a tracked-source claim scan; its exit status is the result. It does *not* invoke `dab/bench/`, which remains quarantined. The older `make reproduce` is a broader artifact workflow and is not the reproduction command for this manuscript's headline evidence.

The manifest `docs/paper/evidence-snapshot.v1.json` is the source for the manuscript macros, `README-AE.md`, toolchain versions, evidence paths, and recorded counts. Its release check requires these generated surfaces to remain byte-synchronous. Test counts are a lower-bound snapshot: a reproduction is successful when the suite is green and is not below 1,434 tests in 172 files, not when it matches a fixed total. The claim scan likewise applies to the tracked source revision (942 scannable files at the recorded revision, zero violations); a normal working-tree scan intentionally changes as generated or untracked files appear.

9 Conclusion

The agent-security debate keeps asking how to make the model trustworthy. Databases stopped asking the analogous question about transactions fifty years ago: assume the client is wrong, isolate its speculation, validate at commit, and log everything. Ghost-Ark is that posture applied to nondeterministic compute — a ledger gate that model checking humbled and then hardened, an OCC discipline honest about its liveness trade, a semantic bound honest about its inputs, and signed `CERTIFIED` receipts for the subset of socket outcomes that the shipped V1 path records. What it refuses to claim is part of the design: a skeptical reviewer should be able to distrust the authors, the README, and the model, and still replay the digest, verify the signature, map each claim to its evidence, and reproduce the failure boundary.

A Artifact Appendix

A.1 Abstract

The artifact is the complete repository: the TypeScript enforcement runtime and receipt schema, the Rust gateway sources, six TLA+ baselines with five seeded mutants and their recorded TLC logs, the malicious-fixture corpus and its metamorphic guard, the verification-cost harness, the claim-language scanner that gates the repository's own documentation (including this manuscript's `.tex` source), and a paper-evidence pipeline that fails closed on E2/E3/E4, TLC, tests, and the tracked-source scan. It is the reproduction path for the evidence cited in Tables 3–4.

A.2 Checklist

- **Program:** Node.js v22.22.3 (TypeScript via native type-stripping), Rust 1.97.1 for the DAB crates, and JDK ≥ 11 for TLC (pinned `tla2tools.jar` v1.7.4).
- **Run-time environment:** Linux x86_64 or macOS arm64; or the provided Ubuntu 22.04 reviewer container (zero host setup beyond Docker).
- **Hardware:** any commodity machine. The paper's latency numbers are from an Apple M1 (8 GB). We have not measured a second host, so we claim no cross-machine reproduction: expect the *ordering* of arms and the baseline ratios to hold, and absolute microseconds not to.
- **Metrics:** stratified detection counts with a control arm, TLC distinct-state counts, verification-cost p50 with IQR, test counts, claim-gate status.
- **Expected wall-clock:** host- and cache-dependent. The `make paper-evidence` report records the duration of each gate; no fixed wall-clock claim is part of the evidence snapshot.
- **Public availability:** the repository is public; an immutable archived snapshot (e.g., Zenodo DOI) is the intended citable artifact.

A.3 Claims mapped to commands

The authoritative map is `README-AE.md` at the repository root; the digest:

- Table 3 (six models clean, five mutants violate, baseline state counts): `bash scripts/run-proofs.sh` $\rightarrow$ generated `artifacts/proofs/proofs_summary.json`; committed raw reference logs are under `proofs/tla/artifacts/` and `proofs/dab/artifacts/`.

- Corpus detection (26/26 verifier-intrinsic, 3/3 control arm, 0 undetected): `npm run experiment:e3`. Exact counts — census provenance.
- That those detections are load-bearing rather than tautological: `npm run experiment:e4`.
- Table 4: `npm run experiment:e2`. The arm ordering and baseline ratios reproduce; absolute microsecond values are machine-dependent, and the IQR is this host's scheduling noise.
- Test snapshot (1,434/172): `npm test`. A green suite must not fall below the snapshot; it may legitimately grow.
- Claim gate (942 tracked source files, 0 violations): run `node tools/research/check-forbidden-claims.mjs` with `-tracked`. The normal working-tree scan is deliberately not compared with this count.
- Semantic-gate bound (§4.2): the focused `semanticAuditReceipt.test.ts` Vitest file.
- Full paper-evidence roll-up: `make paper-evidence`. The container invocation is listed in `README-AE.md`; exit code 0 iff every paper-evidence stage passed.

A.4 The gateway↔verifier round-trip and its enforcement

The Rust gateway and the independent verifier agree on a real ed25519 signature over a domain-separated canonical message including `policy_digest`. A recorded, reproducible round-trip (`dab/roundtrip/`) shows a gateway-emitted `CERTIFIED` receipt verifying, while tamper, mutation, and wrong-key variants are rejected with specific errors. The local socket E2E (`dab/roundtrip/run_socket_e2e.sh`) drives the running gateway with the Rust agent client through a workspace-local Unix socket. It verifies a certified receipt, observes a replay rejection for a reused nonce, and byte-compares the sink's HTTP body with the decoded payload. This is not evidence that the TypeScript IPC client drives the path, that all execution is gateway-mediated, or that the three gates compose at runtime. V1 signs no target and no rejection/abort status; replay and mutation responses are unsigned and malformed or execution-failure paths may emit no receipt. Scope: a local DEV ed25519 key (not KMS/HSM/TPM/Nitro); the bounded tombstone-capacity caveat (§6, item 6) is unchanged.

A.5 What the artifact deliberately does not show

No live-cloud behavior (KMS-mode signing, cloud latency); nothing semantic; no signed rejection receipt; no target-bound DAB V1 receipt; and no runtime OCC or semantic gate. The paper-evidence pipeline publishes its own failures: at earlier commits it exited non-zero on real blockers — including a period when the benchmark's inverted accounting (§5.5) kept the artifact red — and the RESOLVED entries retain that history. An evaluator who wants to test the fail-closed behavior can reintroduce any forbidden phrase into a scanned file and watch the claim stage refuse.

References

[1] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM Trans. Database Syst.*, 6(2), 1981.

[2] N. Shavit and D. Touitou. Software transactional memory. In *PODC*, 1995.

[3] M. Herlihy and J. E. B. Moss. Transactional memory: architectural support for lock-free data structures. In *ISCA*, 1993.

[4] C. H. Papadimitriou. The serializability of concurrent database updates. *J. ACM*, 26(4), 1979.

[5] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil. A critique of ANSI SQL isolation levels. In *SIGMOD*, 1995.

[6] P. A. Bernstein and N. Goodman. Concurrency control in distributed database systems. *ACM Comput. Surv.*, 13(2), 1981.

[7] D. E. Denning. A lattice model of secure information flow. *Commun. ACM*, 19(5), 1976.

[8] A. C. Myers and B. Liskov. A decentralized model for information flow control. In *SOSP*, 1997.

[9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In *AISec*, 2023.

[10] Q. Zhan, Z. Liang, Z. Ying, and D. Kang. InjecAgent: benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of ACL*, 2024.

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: synergizing reasoning and acting in language models. In *ICLR*, 2023.

[12] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa. Firecracker: lightweight virtualization for serverless applications. In *NSDI*, 2020.

[13] NVIDIA. NemoClaw architecture overview. `https://docs.nvidia.com/nemoclaw/latest/`, 2026. Accessed 2026-07-16.

[14] B. Laurie. Certificate transparency. *Commun. ACM*, 57(10), 2014.

[15] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[16] Y. Yu, P. Manolios, and L. Lamport. Model checking TLA+ specifications. In *CHARME*, 1999.

[17] C. Newcombe, T. Rath, F. Zhang, B. Munteanu, M. Brooker, and M. Deardeuff. How Amazon Web Services uses formal methods. *Commun. ACM*, 58(4), 2015.

[18] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on test data selection: help for the practicing programmer. *IEEE Computer*, 11(4), 1978.

[19] R. B. Nelsen. *An Introduction to Copulas*, 2nd ed. Springer, 2006.

[20] G. Boole. *An Investigation of the Laws of Thought*. Walton and Maberly, 1854.

[21] J. A. Hanley and A. Lippman-Hand. If nothing goes wrong, is everything all right? Interpreting zero numerators. *JAMA*, 249(13), 1983.